UML Design Document

Fresh Mart: Grocery Shop Management System

OOPS Lab Mini Project | Submitted to: Dr. Sakshi Ranjan

Team: R025 Anmol Jain, R028 Thakarshi Jayandhan, R031 Harsh Joshi

GitHub repository: [paste your repository link]

1. Purpose

This document records the design of the Fresh Mart system before and alongside its Java implementation. The system lets a customer view products, add them to a cart, and check out. Stock and input checks prevent invalid purchases. It runs as a console program and as a website served by Java's built-in HTTP server.

2. Diagrams in this document

No.	Diagram	What it shows
1	Class diagram	Classes, interface, inheritance and all relationships
2	Use case diagram	What the customer can do with the system
3	Sequence diagram	Add to cart and checkout, step by step, on the website
4	Activity diagram	The shopping flow with every validation decision
5	Deployment diagram	How the browser and the Java server run together on one laptop

3. Relationships used

Relationship	Classes	Meaning
Realization	Product implements Purchasable	Every product must provide price, availability and purchase().
Inheritance	FoodItem, BeverageItem, NonFoodItem extend Product	Subclasses reuse name, price and quantity and add their own fields.
Composition	Customer has a Cart; Cart contains CartItem	The cart and its lines cannot exist without their owner.
Aggregation	GroceryStore holds Products	Products exist independently of one store object.
Association	CartItem refers to Product	Each cart line points to the product being bought.
Dependency	GroceryShop creates GroceryStore; GroceryStore creates Customer	One class uses another for a short time or to build it.

4. Class diagram

Product is abstract and implements the Purchasable interface. FoodItem, BeverageItem and NonFoodItem extend it and each overrides displayDetails(), getType() and getInfo(), which is polymorphism. Customer owns a Cart (composition), a Cart holds CartItem lines, and GroceryStore keeps the inventory of Products (aggregation). GroceryWeb exposes the same store to the browser, keeping one Customer per visitor session.

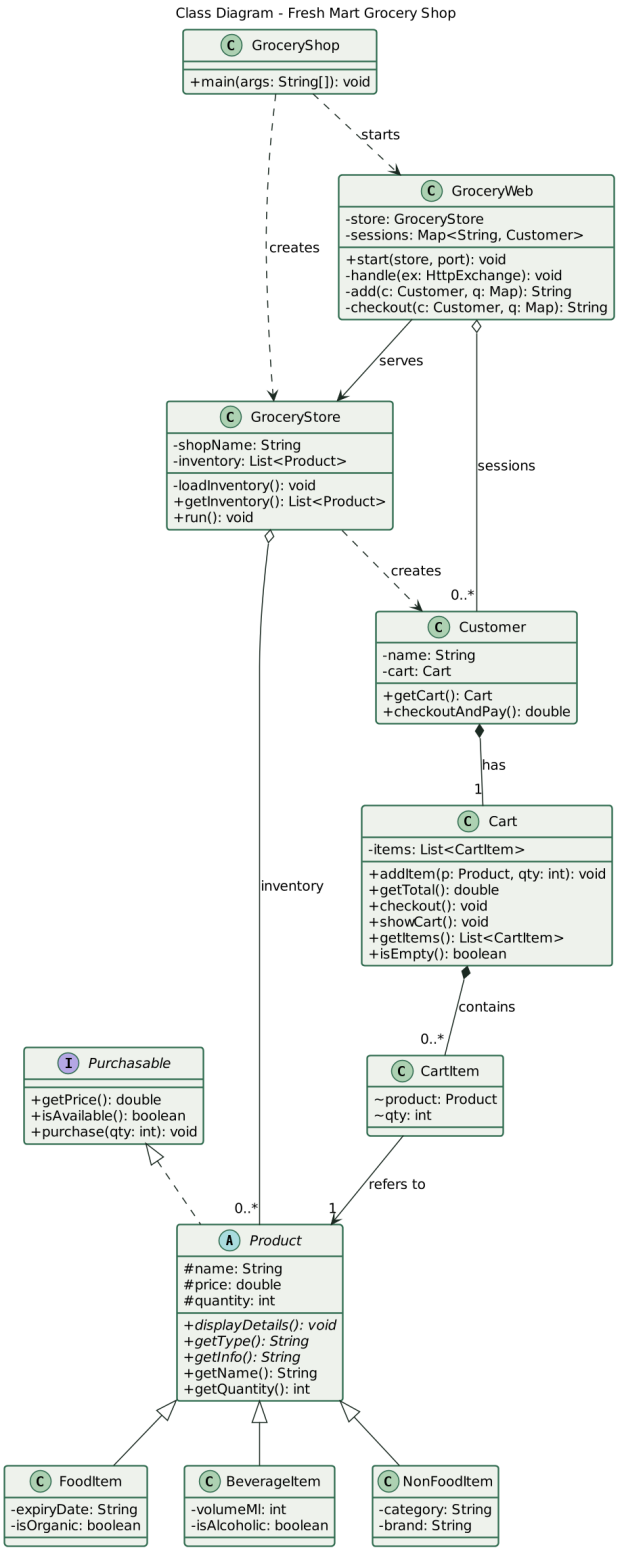


Figure 1: Class diagram

5. Use case diagram

The customer is the only actor. Adding to cart and checking out both include validation of quantity and stock, and checkout also updates stock.

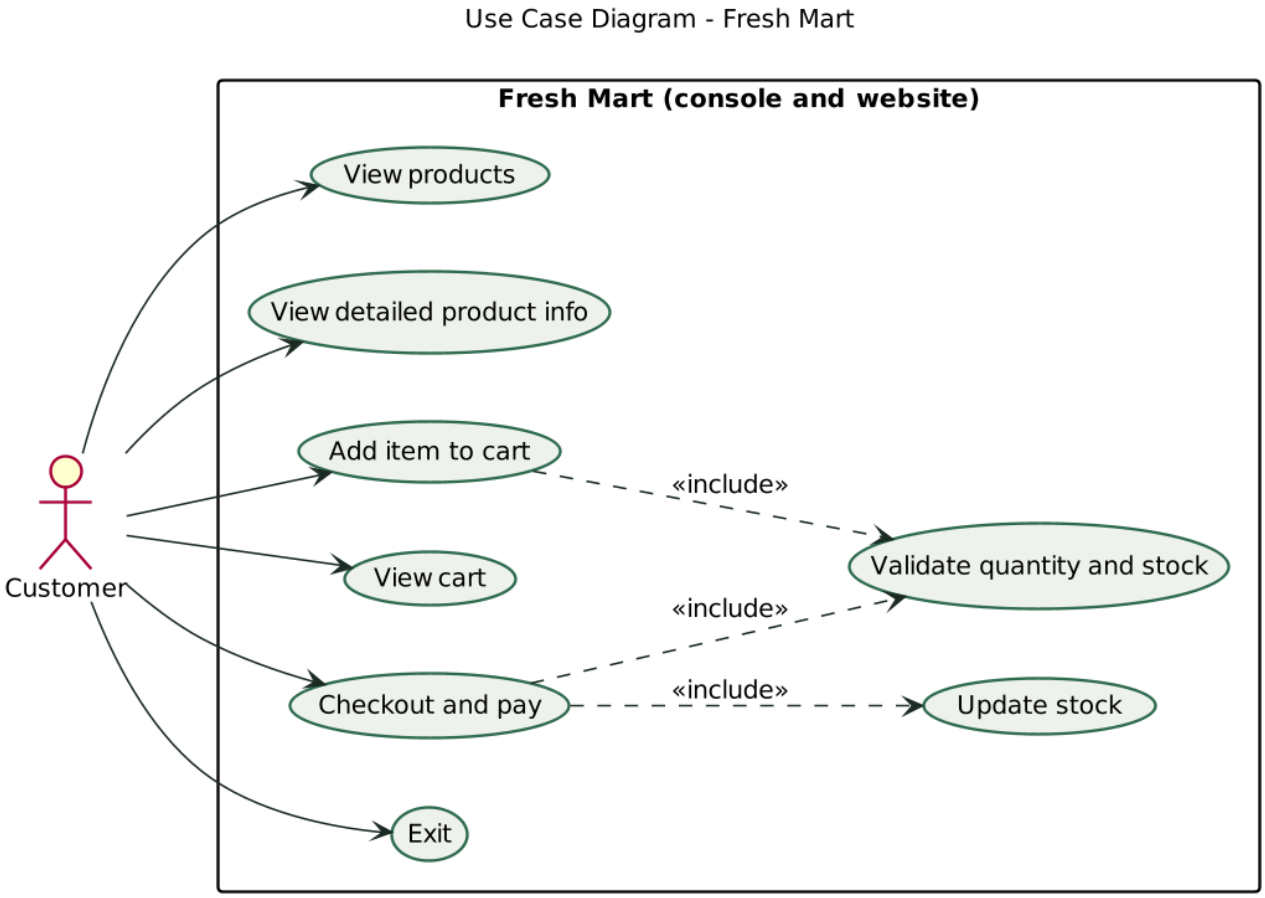


Figure 2: Use case diagram

6. Sequence diagram

Shows the website flow. Add to cart validates the id, quantity and stock before calling Cart.addItem(). Checkout re-checks stock for every line, then Cart.checkout() calls Product.purchase() to reduce quantity and clears the cart.

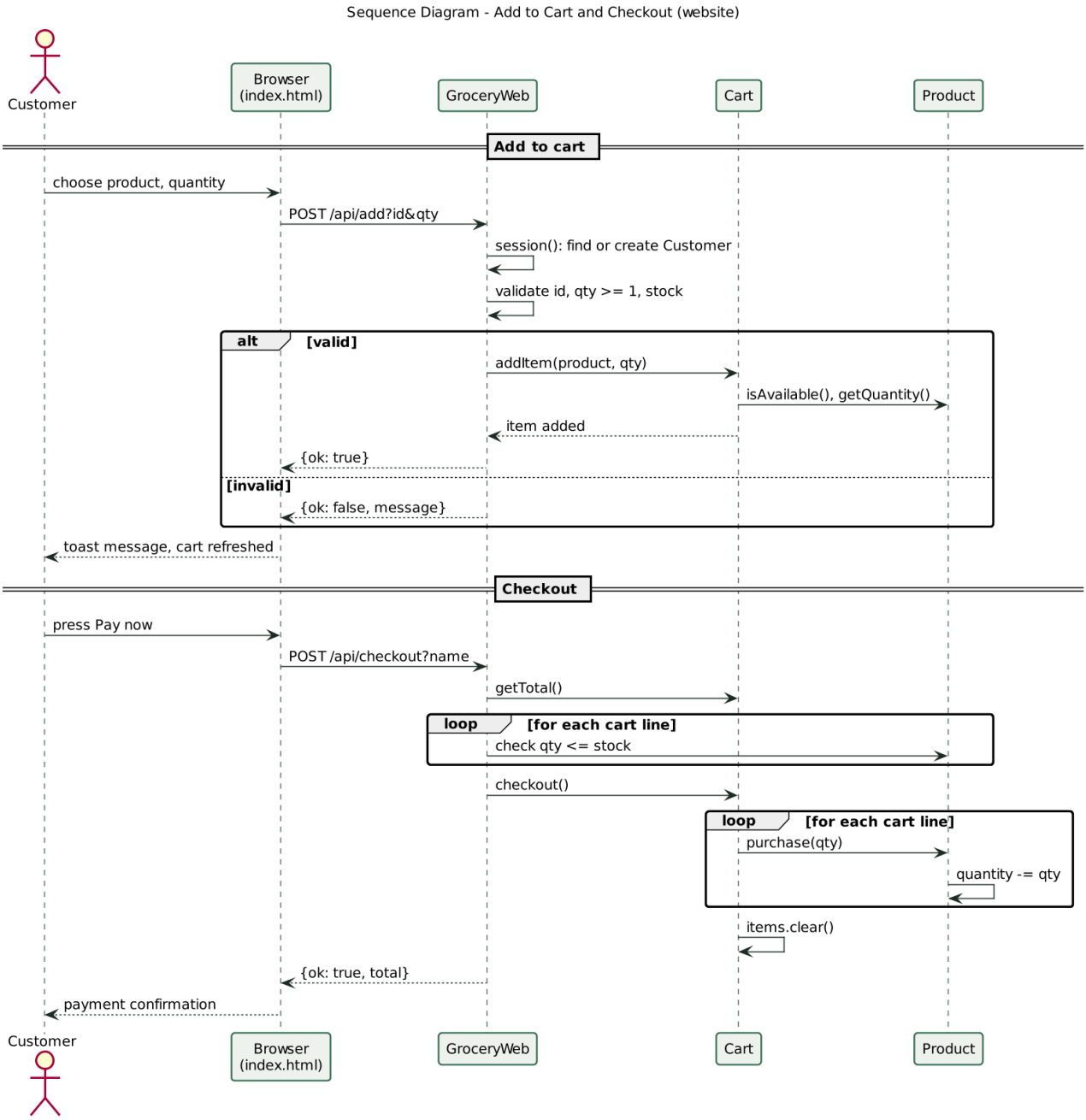


Figure 3: Sequence diagram

7. Activity diagram

Follows one shopping session from loading the inventory to payment, with the decisions for valid input, stock and an empty cart.

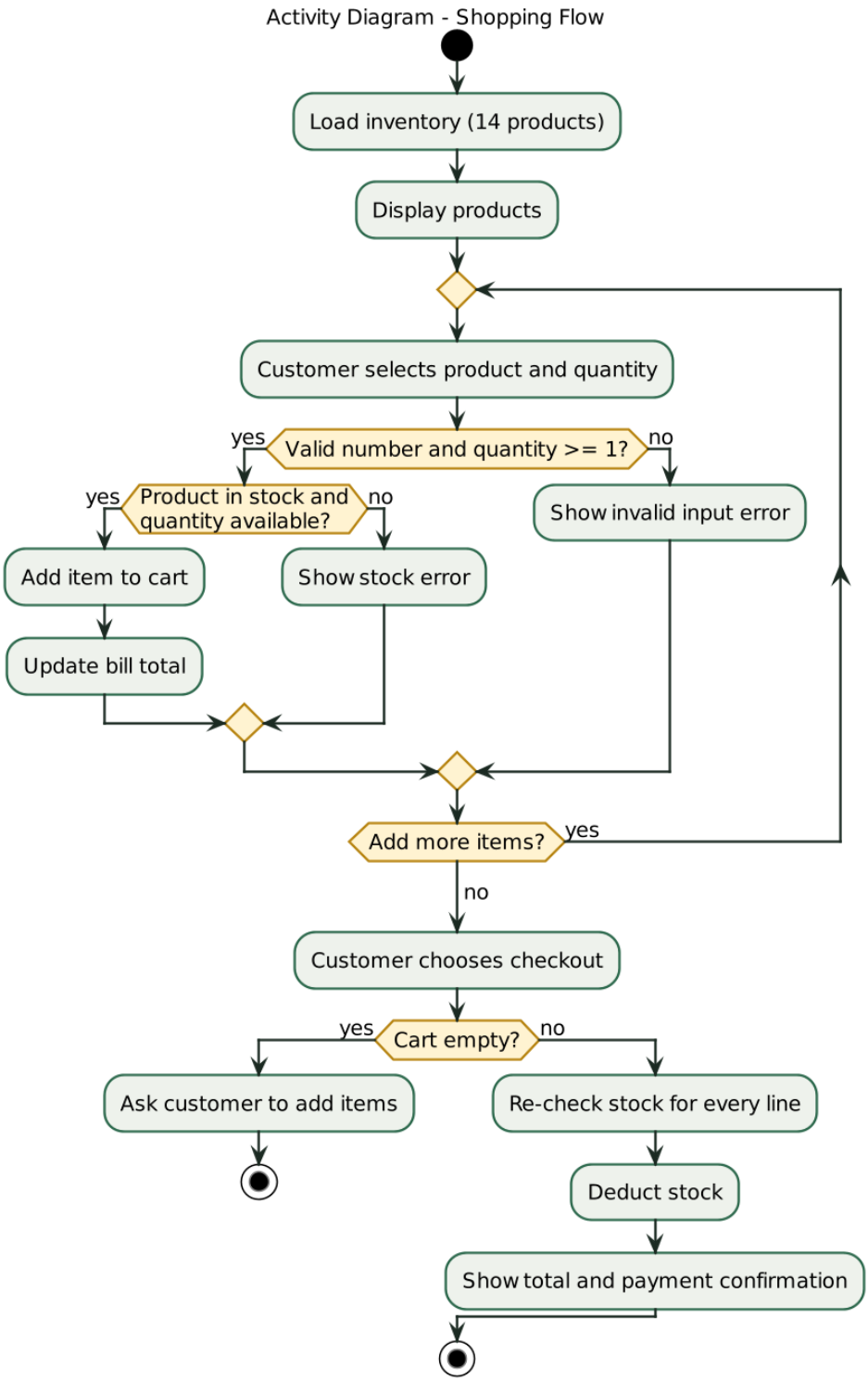


Figure 4: Activity diagram

8. Deployment diagram

The browser and the Java server run on the same laptop. The browser sends requests to localhost on port 8080, and the server answers with the web page and JSON data from the store objects.

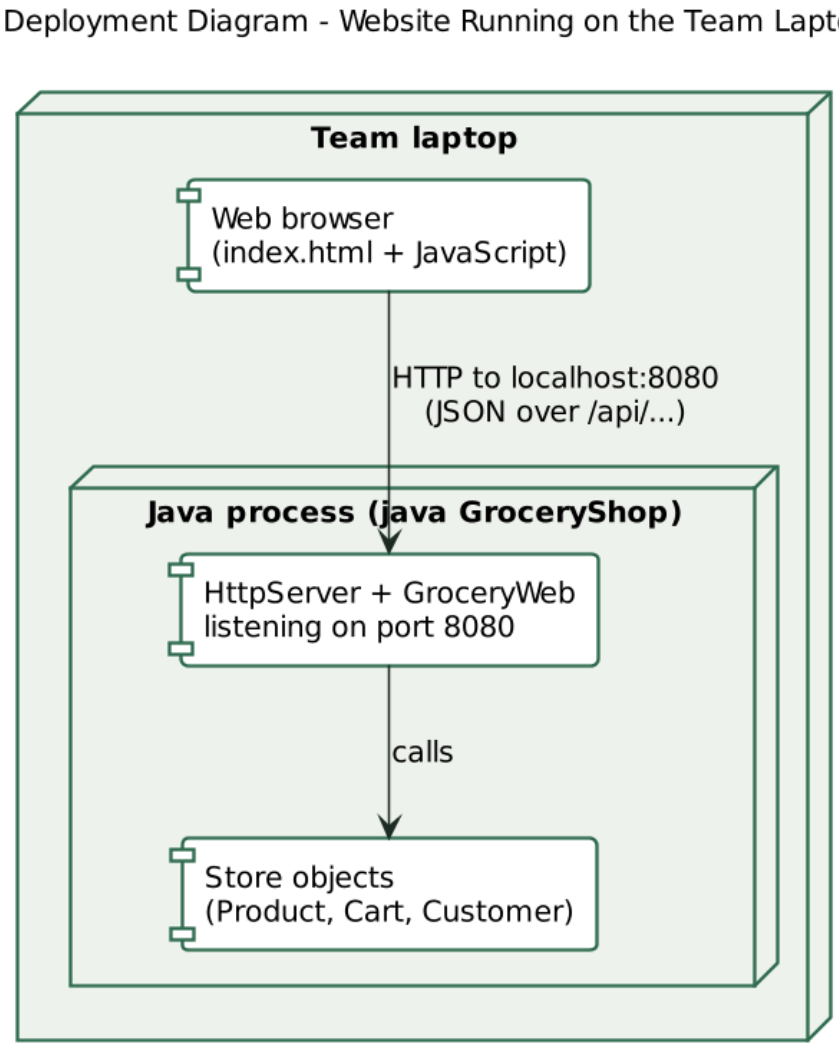


Figure 5: Deployment diagram